

Securing Language Models Against Vulnerability Encoding

Rui Melo

FEUP & Fraunhofer Portugal

1. Why it Matters

LLMs boost productivity, but **often generate insecure code**

Root Cause: LLMs learn from both secure and insecure code. Current defences (e.g., static analysis) are **reactive**, not preventive.

```
void vulnerable_function(){  
    char buffer[20];  
    scanf("%s", buffer);  
}
```

Buffer
Overflow

Prevalence of Unsafe Code

GitHub Copilot - October 2023

Python

29,50%

Java

24,20%

43 different CWEs

29.50% of Python and 24.20% of Java code produced by Copilot is vulnerable

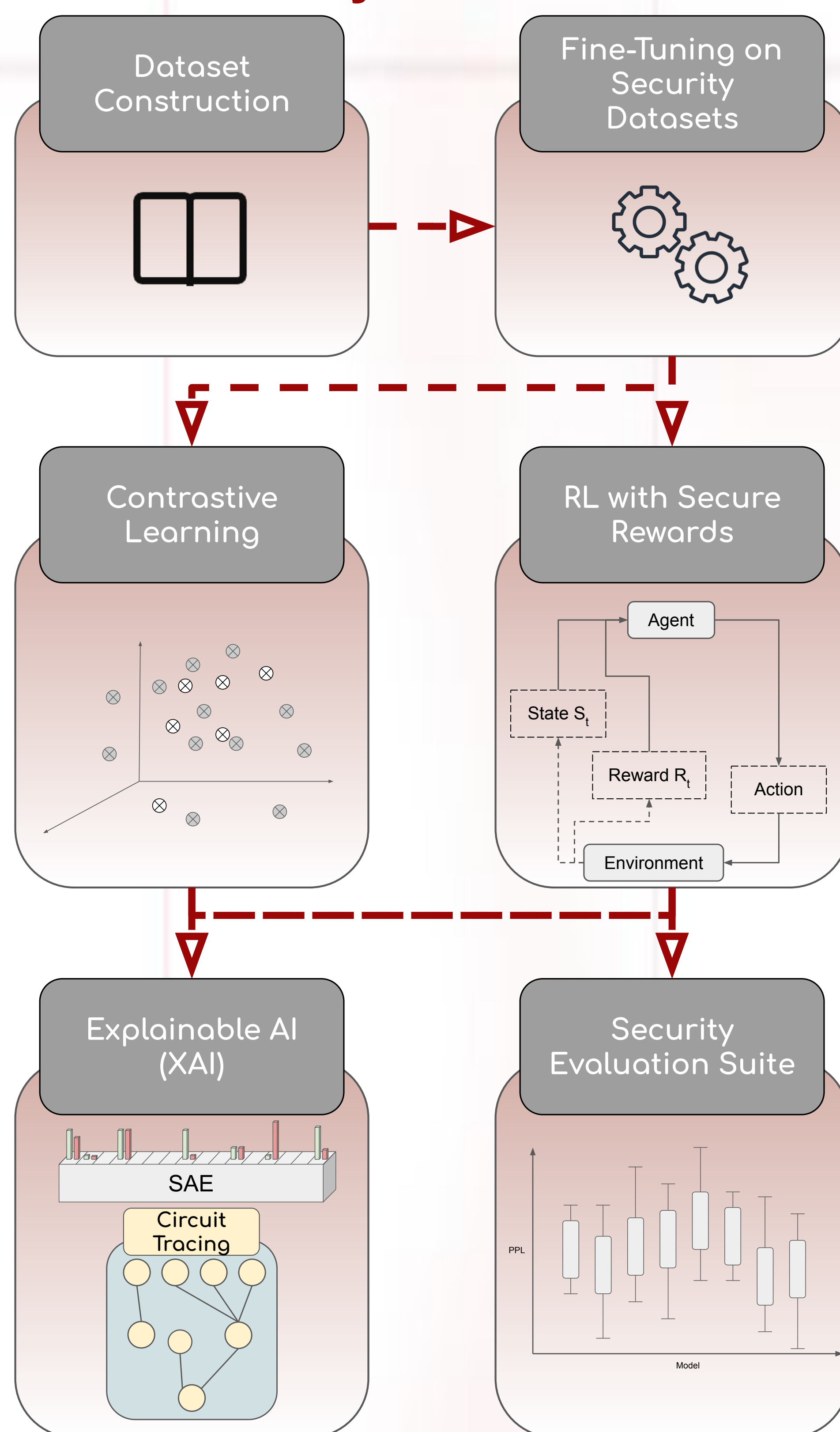
2. Security in AI Code Generation

LLMs trained on open-source code **inherit both good and bad practices**.

Static / dynamic analysis helps, but lacks full coverage.

Need: Train models that **understand and avoid** vulnerabilities by design.

3. Our Security-Aware Framework



4. Conclusion & Outlook

Security must be embedded into AI coding tools, not patched afterwards.

We hypothesise that our approach can:

- **Reduce vulnerabilities** in LLM-generated code
- Enhance **developer trust** and **model transparency**
- Support **secure-by-design** AI programming workflows